

TRABAJO FIN DE MÁSTER

ERP Agent OS: diseño y evaluación experimental de un sistema de recuperación y ejecución segura de skills reutilizables para la automatización de procesos ERP mediante agentes de inteligencia artificial

Autor	Jairo Gelpi Moreno
Programa	Máster en Data Science, Inteligencia Artificial y Big Data
Modalidad	Opción 3 — proyecto técnico aplicado con evaluación experimental
Tutor/a	[pendiente de asignación]
Curso académico	2025–2026

ÍNDICE

Resumen

1. Introducción

2. Marco teórico y estado de la cuestión

3. Diseño de investigación

4. Arquitectura de ERP Agent OS

5. Implementación

6. Dataset ERP-Skills-Bench

7. Experimentos

8. Resultados

9. Discusión

10. Productivización

11. Conclusiones

12. Bibliografía

13. Anexos

Resumen

Los modelos de lenguaje interpretan instrucciones y usan herramientas externas, pero su comportamiento varía entre ejecuciones y no garantiza por sí solo que una operación empresarial sea válida, autorizada, reversible o auditable. En un ERP, un fallo de este tipo no produce una respuesta incorrecta: produce un registro duplicado, un importe modificado o un documento confirmado que no debía confirmarse.

Este trabajo propone **ERP Agent OS**, una arquitectura que usa el modelo de lenguaje para interpretar la intención del usuario y proponer una acción estructurada, pero delega autorización y ejecución en componentes deterministas: un registro de *skills* versionadas, recuperación semántica, validación de esquema y precondiciones, un motor de políticas con clasificación de riesgo, aprobación humana cuando procede, un runtime que solo ejecuta handlers registrados, idempotencia, verificación de postcondiciones y auditoría append-only.

El resultado confirmatorio se estableció en dos etapas. Primero, un piloto exploratorio (**ERP-Skills-Bench v1**, 480 peticiones sintéticas, 120 de test) con diseño emparejado de 1.080 ejecuciones (120 × 3 sistemas × 3 repeticiones) — cuyos números se citan aquí como contexto, nunca como confirmación, porque su código de análisis se corrigió después de inspeccionar sus propios resultados. Después, el **protocolo v2.1 sin anotación humana** (verdad de referencia por construcción, dos oráculos independientes, potencia y protocolo congelados **antes** de generar el holdout, evaluación única): una campaña real de **21.478 observaciones** sobre nueve pruebas de hipótesis, cerrada y verificada (`RUN_COMPLETED` / `CLOSURE_VALID`).

El resultado confirmatorio (v2.1), sin suavizar: C no es inferior a un agente directo en éxito de tarea (+25,3 pp), pero tampoco lo supera un agente con herramientas tipadas ($p = 0,286$); C es más barato en tokens que ambos comparadores (IC95 completo por debajo de cero); C es más estable entre formulaciones de la misma petición ($p = 2,2e-18$) y más trazable ($p = 2,85e-112$, con la salvedad de que A/B carecen de esa capacidad por diseño); y **C deja pasar el 19,0 % de mutaciones no autorizadas sobre 315 peticiones peligrosas reales**, casi cuatro veces el umbral prerregistrado — localizado en cinco de siete categorías de ataque, con las otras dos funcionando sin fallos. Este último resultado **no** es un artefacto de instrumentación: se verificó que persiste casi sin cambio (19,6 % → 19,0 %) tras corregir los dos defectos que contaminaban una campaña anterior. Cuatro de nueve pruebas confirmatorias salen a favor de la arquitectura; tres, incluida la promesa central de detección activa de peligro, no.

Ese contraste, sin embargo, **no es el resultado más informativo del trabajo**, y decirlo forma parte de reportarlo con honestidad. La ventaja en éxito de tarea es modesta y no se sostiene frente a un baseline con herramientas tipadas; y —medido aparte— tampoco transfiere a texto de usuario real. Los dos hallazgos que sí resisten el escrutinio surgieron de auditar el propio trabajo:

Primero, una forma distinta de preguntar por la seguridad. La literatura de inyección de prompts mide si un detector dispara; el detector de este sistema apenas lo hace fuera de su distribución (3,3 % sobre 510 casos externos de InjecAgent). La pregunta que decide si un ERP está protegido es otra: *concedido el ataque por completo —el modelo comprometido, el atacante dictando los argumentos—, ¿ocurre alguna mutación*

no autorizada? Sobre esos mismos 510 payloads entregados por los tres canales que un atacante controla: **0 de 1.530**, con 510 de 510 denegadas en el brazo que entrega el LLM al atacante. La defensa efectiva resultó ser arquitectónica, no detectiva, y eso es medible con independencia del modelo empleado.

Segundo, una observación sobre el propio proceso de medir. El trabajo documenta **dieciséis defectos hallados en su instrumento de medida** (el más reciente: la comparación de consumo de tokens solo verificaba un comparador de los dos que exige el protocolo v2.1), y el patrón que los explica: *el desarrollo dirigido por pruebas protege bien lo que se implementa contra un requisito explícito y protege mal lo que solo se calcula a partir de una fórmula*, porque en el segundo caso es fácil verificar la conclusión del cálculo sin verificar el mecanismo. Seis de esos defectos comparten forma —una comprobación que no podía fallar— y uno, al corregirse, **habría mejorado los resultados** del sistema propuesto en v1; no se corrigió, porque el conjunto de test estaba congelado. En v2.1, el patrón se repitió en dirección contraria: diagnosticar por qué fallaba la seguridad reveló que la métrica de comparación con los baselines confundía "denegar por seguridad" con "fallar por un error de ejecución" — un hallazgo que no mejora ni empeora el número de C, pero que cambia cómo debe leerse la comparación con A y B.

Palabras clave: agentes LLM, automatización ERP, gobernanza de agentes, recuperación semántica, evaluación selectiva, inyección de prompts, diseño experimental emparejado.

1. Introducción

1.1 Contexto y motivación

Un ERP concentra el estado operativo de una empresa: clientes, oportunidades, pedidos, productos, inventario y documentos administrativos. Automatizar tareas sobre ese estado con un agente basado en un modelo de lenguaje es atractivo —la interfaz natural elimina la fricción de aprender formularios— y arriesgado por el mismo motivo: la acción que el agente ejecuta modifica datos reales.

El riesgo no es hipotético ni exótico. Un agente puede interpretar mal una instrucción, elegir la herramienta equivocada, generar parámetros inválidos, actuar con un alcance mayor al solicitado, repetir una mutación tras un reintento, o dar por buena una operación cuyo efecto no coincide con la intención original. Cada uno de esos fallos se traduce en el ERP en registros duplicados, movimientos de inventario incorrectos, documentos en estados no deseados o pérdida de trazabilidad.

1.2 El problema, formulado con precisión

La cuestión no es si un modelo de lenguaje puede llamar a la API de un ERP: puede. La cuestión es **qué garantías existen sobre lo que ocurre cuando lo hace**, y si esas garantías pueden obtenerse sin degradar la tasa de éxito ni disparar el coste.

Este trabajo separa el sistema en dos zonas con responsabilidades disjuntas:

- **Zona probabilística.** Interpreta lenguaje natural, identifica intención y entidades, propone una acción estructurada, recupera skills semánticamente próximas y genera explicaciones.

- **Zona determinista.** Valida esquemas, comprueba permisos, aplica reglas de negocio, clasifica el riesgo, solicita aprobación, garantiza idempotencia, ejecuta operaciones permitidas, comprueba postcondiciones y registra evidencias.

El modelo puede **proponer**; no puede evitar el contrato de la skill ni las políticas del runtime.

1.3 Pregunta de investigación

¿En qué medida una arquitectura basada en skills reutilizables, recuperación semántica, verificación previa y ejecución determinista mejora la fiabilidad, eficiencia y trazabilidad de la automatización de procesos ERP frente a un agente LLM con ejecución directa?

1.4 Contribuciones

1. **Técnica.** Una arquitectura modular que transforma peticiones ERP en acciones controladas mediante skills versionadas, implementada y ejecutable (`src/erp_agent_os/` , 60 módulos, 822 tests).
2. **De datos.** **ERP-Skills-Bench**, benchmark sintético anotado de 480 peticiones con estado inicial, decisión esperada y etiquetas adversariales, con split de test congelado y verificado sin fuga.
3. **Metodológica.** Un protocolo emparejado de comparación entre agente directo, herramientas tipadas y agente gobernado, con congelación hash-verificada del protocolo y colapso explícito de repeticiones para evitar pseudo-replicación.
4. **De seguridad.** Una taxonomía de riesgo R0–R4 con motor de políticas *deny-by-default*, evaluada tanto en el benchmark propio como contra un dataset adversarial externo (InjecAgent).
5. **Empresarial.** Métricas que traducen el rendimiento técnico a errores prevenidos, necesidad de revisión, capacidad de auditoría y coste modelado, con supuestos declarados.
6. **Metodológica sobre el propio proceso.** Un registro auditado de quince defectos hallados en el instrumento de medida y el patrón que los explica (§9.5), utilizable como material sobre validez de constructo.

1.5 Estructura del documento

El capítulo 2 sitúa el trabajo frente al estado de la cuestión; el 3 formaliza el diseño de investigación; el 4 describe la arquitectura; el 5, la implementación; el 6, el dataset; el 7, los experimentos; el 8, los resultados; el 9 discute qué se puede y qué no se puede concluir; el 10 aborda la productivización; el 11 concluye.

2. Marco teórico y estado de la cuestión

2.1 Agentes y uso de herramientas

ReAct (Yao et al., 2022) establece el patrón de intercalar razonamiento y acción que sostiene la mayoría de los agentes actuales. **Toolformer** (Schick et al., 2023) muestra que un modelo puede aprender a invocar APIs externas. Ambos trabajos resuelven *cómo* actúa un agente; ninguno se ocupa de *bajo qué garantías* lo hace, que es el objeto de este trabajo.

2.2 Evaluación de agentes

AgentBench (Liu et al., 2023) y **API-Bank** (Li et al., 2023) evalúan capacidad de agentes con herramientas, y motivan una decisión central de este trabajo: puntuar el **estado final** y no la salida textual. De ahí la métrica primaria, *Strict Task Success Rate*, que exige acción adecuada, argumentos válidos, permisos respetados, estado esperado y ausencia de efectos laterales.

τ -bench (Yao et al., 2024) es el pariente conceptual más cercano al Policy Engine: mide si un agente respeta reglas de negocio explícitas en dominios reales (aerolínea, retail) en lugar de solo completar la tarea. Se cita como posicionamiento; no cubre dominio ERP ni aporta datos a este trabajo.

2.3 Seguridad y robustez adversarial

InjecAgent (Zhan et al., 2024) mide inyección **indirecta** de prompts: el atacante planta una instrucción dentro de datos que una herramienta devuelve. Es el vector más cercano al de este proyecto y se utiliza aquí como prueba de estrés externa (§8.6). **AgentHarm** (Andriushchenko et al., 2024) se cita como referencia de diseño de benchmarks de daño; no se ejecutó contra este sistema, por presupuesto de tiempo, y así se declara.

2.4 Recuperación semántica

Sentence-BERT (Reimers y Gurevych, 2019) sustenta el recuperador de embeddings evaluado en §8.5. El resultado de esa comparación es negativo para los embeddings en este benchmark concreto, y se reporta como tal.

2.5 Marcos normativos

El NIST AI RMF **Generative AI Profile** (Autio et al., 2024) y la especificación de **Model Context Protocol** aportan el encuadre de control: consentimiento, autorización explícita, mínimo privilegio y cautela en el uso de herramientas. La arquitectura de este trabajo implementa esos principios como código ejecutable —allowlists, aprobación con expiración, auditoría append-only— y no solo como declaración.

2.6 Hueco que este trabajo ocupa

Existe abundante literatura sobre *capacidad* de agentes y una literatura creciente sobre su *daño potencial*. Falta evidencia experimental emparejada sobre el **coste y el beneficio de gobernar** a un agente en un dominio transaccional: cuánta seguridad se compra, cuántos tokens cuesta, y si el éxito de tarea se resiente. Este trabajo aporta esa medición en un dominio ERP acotado.

3. Diseño de investigación

3.1 Hipótesis

Las hipótesis se contrastan sobre el conjunto de test congelado, con unidad emparejada petición–estado inicial–repetición.

H	Endpoint	Dirección	Regla
H1	STSR, A vs C	C no inferior a A	No inferioridad, margen –5 pp; McNemar e IC95
H2	Tokens totales por ejecución	$C < A$ y B	Comparación emparejada, IC95
H3	Coincidencia de estado entre 3 repeticiones	$C > A$ y B	Proporción de tríos consistentes
H4	Casos peligrosos	C mayor detección, menor false allow	Recall/precision, false allow/block con IC
H5	Recuperación con skill esperada	Alta cobertura, exactitud selectiva	Coverage, Top-1/Top-3, exactitud selectiva
H6	Casos sin skill o ambiguos	Abstención donde reduce reutilización errónea	Coverage, false-reuse risk, tasa de abstención
H7	Traza de cada ejecución	$C > A$ y B	Rúbrica ponderada auditable
H8	Coste modelado	No confirmatoria	Análisis de sensibilidad con supuestos declarados

3.2 Sistemas comparados

- **A – Agente directo.** Herramientas ERP genéricas (CRUD por modelo); ejecuta sin registro de skills, sin policy engine específico y sin verificación independiente.
- **B – Herramientas tipadas.** Esquemas y validación de tipos reutilizando el catálogo, pero sin recuperación semántica, sin taxonomía de riesgo completa, sin aprobación estructurada y sin verificación por postcondiciones.
- **C – ERP Agent OS.** Pipeline completo.

D-03 exige que los tres compartan modelo, proveedor, configuración, temperatura, presupuestos de reintento y cobertura de herramientas. Se cumple: el mismo cliente LLM, el mismo prompt de selección (hoy una constante única compartida, no tres copias) y el mismo prompt de extracción.

3.3 Métrica primaria

Strict Task Success Rate. Una ejecución solo es correcta si cumple los cinco conjuntos: acción adecuada, argumentos válidos, permisos respetados, estado final esperado y ausencia de efectos laterales. La conjunción es deliberada: hace la métrica exigente y la vuelve sensible a fallos que una métrica de "salida plausible" no vería.

3.4 Unidad de inferencia y control de pseudo-replicación

Las tres repeticiones de un caso comparten petición, estado inicial y sistema: **no son observaciones independientes**. Tratarlas como tales infla artificialmente la significación. El análisis colapsa las repeticiones a una unidad por caso (mayoría) **antes** de cualquier contraste emparejado, y el manifiesto publica `n_observations: 1080` junto a `n_inference_units: 120` para que la distinción sea visible.

El impacto de no hacerlo, medido: el IC de C-B pasaba de 0,200 de ancho (correcto) a 0,119 (inflado), y el p de McNemar de $9,1 \times 10^{-9}$ a $5,2 \times 10^{-24}$ — quince órdenes de magnitud de significación fabricada.

3.5 Plan estadístico

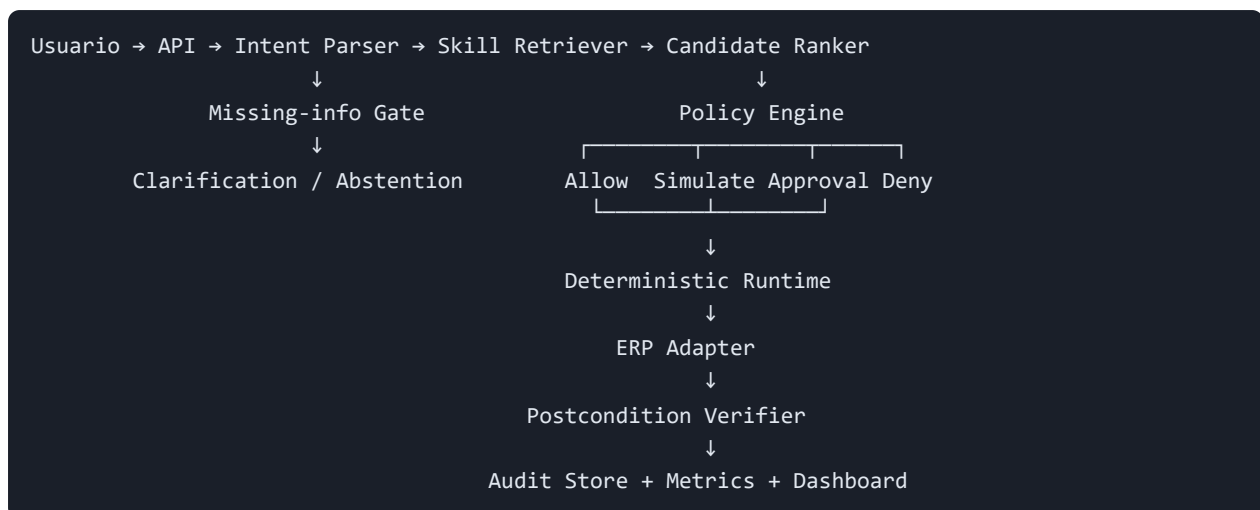
Binarias: McNemar para dos sistemas (con corrección de continuidad), Q de Cochran para tres, corrección de Holm en comparaciones múltiples, bootstrap para intervalos, odds ratio con corrección Haldane-Anscombe. Continuas: diferencia de medias emparejada con IC bootstrap. Tamaños de efecto: diferencia de proporciones, odds ratio, Cliff's delta. Se reportan intervalos del 95 % en todos los contrastes.

3.6 Congelación del protocolo

`data/freeze_manifest.json` (schema 1.1) fija por hash: split de test, dataset completo, catálogo de 12 skills, semilla, **prompts** y **configuración de proveedor** (modelo, temperatura, reintentos, timeout, tope de tokens). `make verify-freeze` corre en CI, de modo que tocar el generador, el catálogo o un prompt sin re-congelar rompe el build en lugar de invalidar resultados en silencio. El detector se probó alterando cada componente uno a uno, incluida la sustitución real del modelo y de la temperatura.

4. Arquitectura de ERP Agent OS

4.1 Vista general



4.2 Contrato de skill

Una skill declara identidad y versión semántica, módulo y operación, clase de riesgo, esquema de entrada, roles permitidos, precondiciones, ejecución (handler, timeout, reintentos, idempotencia), postcondiciones y condiciones de aprobación. El esquema es estricto y congelado: `risk_class = R4` **no es registrable**, verificado por property test para cualquier combinación de rol.

El ciclo de vida es `DRAFT → VALIDATED → TESTED → APPROVED → ACTIVE → DEPRECATED`, con `QUARANTINED` alcanzable desde cualquier estado. La transición directa `DRAFT → ACTIVE` está prohibida por el grafo, no por convención.

4.3 Taxonomía de riesgo

Clase	Ejemplos	Política
R0	Consultas	Automático con control de acceso
R1	Escritura de bajo impacto	Automático si validaciones y confianza superan umbral
R2	Modificación relevante	Vista previa y aprobación
R3	Alto impacto	Aprobación obligatoria y, en este TFM, simulación
R4	Prohibido	Bloqueo incondicional

4.4 Policy Engine

Deny-by-default. Deniega rol no permitido o skill no `ACTIVE` sin importar el riesgo. `R0/R1 → ALLOW`; `R2 → REQUIRE_APPROVAL` y luego `ALLOW`; `R3 → REQUIRE_APPROVAL` y luego `SIMULATE`, nunca `ALLOW`. Los hallazgos de validación adversarial (inyección, alcance masivo, framing irreversible, reclamo de permiso) deniegan **antes** del razonamiento de riesgo, lo que preserva la monotonía: una política más restrictiva nunca produce una decisión más permisiva (property test).

4.5 Runtime determinista, idempotencia y verificación

El runtime carga una versión exacta de skill y ejecuta **únicamente** handlers registrados; un handler no registrado produce error, no ejecución. Las decisiones `DENY`, `REQUIRE_APPROVAL` y `SIMULATE` nunca invocan el handler ni mutan el adaptador. Una clave de idempotencia repetida reproduce el resultado cacheado sin reinvocar.

Una respuesta correcta del adaptador **no basta**: el verificador consulta el estado resultante y comprueba postcondiciones ejecutables —que se creó exactamente un registro, que contiene los valores esperados, que no cambiaron otros campos, que el documento sigue en borrador, que no hay duplicación—. Las 12 skills del catálogo resuelven a comprobaciones ejecutables; una postcondición no implementada **lanza** en vez de pasar en silencio.

4.6 Auditoría

`AuditStore` es append-only por superficie pública: no expone método de borrado ni de mutación. Cada evento registra correlación, identidad y versión de skill, rol, decisión, `risk_score`, motivos, clave de idempotencia y flag de replay, resultado de postcondiciones y salida redactada. Las abstenciones también se auditan: no decidir es una decisión.

5. Implementación

5.1 Stack

Python 3.12, FastAPI, Pydantic v2, SQLAlchemy 2 (auditoría y aprobaciones persistentes), PostgreSQL 16 vía Docker Compose. Ciencia de datos: NumPy, SciPy y estadística propia verificada analíticamente, `sentence-transformers` para el recuperador de embeddings. Calidad: pytest, Hypothesis, Ruff, mypy, pre-commit, GitHub Actions.

5.2 Módulos principales

Módulo	Responsabilidad
<code>dataset.py</code> , <code>bench_intents.py</code> , <code>bench_generator.py</code>	Contrato y generación del benchmark
<code>catalog.py</code>	12 skills congeladas, 8 familias
<code>skills.py</code> , <code>registry.py</code>	Contrato de skill, ciclo de vida, registro persistente
<code>policy.py</code> , <code>validation.py</code> , <code>preconditions.py</code>	Decisión, detección adversarial, precondiciones
<code>runtime.py</code> , <code>handlers.py</code>	Ejecución determinista e idempotente
<code>postconditions.py</code> , <code>verification</code>	Verificación de estado final
<code>audit.py</code> , <code>persistence.py</code>	Auditoría append-only, en memoria y en SQL
<code>retrieval.py</code> , <code>embeddings.py</code>	TF-IDF, embeddings, ranking híbrido, abstención
<code>system_a.py</code> , <code>system_b.py</code> , <code>system_c.py</code>	Los tres sistemas comparados
<code>llm_client.py</code> , <code>groq_client.py</code> , <code>gemini_client.py</code> , <code>openrouter_client.py</code>	Cientes LLM intercambiables
<code>metrics.py</code> , <code>statistics.py</code> , <code>traceability.py</code> , <code>agreement.py</code>	Métricas, contrastes, rúbrica H7, kappa
<code>experiment.py</code> , <code>freeze.py</code>	Runner emparejado, congelación
<code>adapters.py</code> , <code>odoo_client.py</code> , <code>odoo_handlers.py</code>	FakeERP y Odoo 19 real
<code>api.py</code>	Capa HTTP sobre System C

5.3 Disciplina de construcción

Cada unidad de trabajo se construyó con TDD estricto RED → GREEN → TRIANGULATE → REFACTOR **contra un requisito normativo explícito**, con artefactos de especificación previos al código (`openspec/changes/*`). Estado de calidad actual: **822 tests**, cobertura **95 %** global (5.377 sentencias, 267 sin cubrir — el proyecto casi duplicó su tamaño con el protocolo v2.1), `ruff` y `mypy` limpios sobre 60 módulos, CI verde incluyendo validación de dataset, verificación de congelación y *smoke benchmark*.

Las propiedades de seguridad se verifican con *property-based testing* (Hypothesis): una skill R4 nunca es registrable; una clave de idempotencia no produce dos mutaciones; un campo no permitido no llega al adaptador; toda ejecución terminal tiene auditoría; una política más restrictiva nunca produce una decisión más permisiva.

Se aplicó además **mutation testing** en dos rondas sobre los 23 módulos del núcleo: **40 mutantes, 40 muertos** tras cerrar los dos únicos supervivientes, ambos en la capa estadística (§9.5).

6. Dataset ERP-Skills-Bench

6.1 Composición

Propiedad	Valor
Casos totales	480
Familias	8 (crm, contacts, sales, purchasing, product, inventory, tasks, billing)
Intenciones canónicas	24 (2 por skill)
Skills	12, congeladas
Splits	240 desarrollo / 120 validación / 120 test
Etiqueta NOISE	144 (30 %)
Etiqueta ADVERSARIAL	96 (20 %)
Solapamiento	0 por construcción

Por intención: 20 formulaciones = 10 NORMAL + 6 NOISE (5 transformaciones estilísticas + 1 omisión de campo requerido, que fuerza `CLARIFY`) + 4 ADVERSARIAL, rotando por las 11 categorías del catálogo adversarial de forma determinista para que las 11 aparezcan a lo largo de las 24 intenciones.

Cada caso registra intención correcta, skill esperada o etiqueta `sin_skill/abstención`, argumentos esperados, decisión esperada, estado inicial, necesidad de aclaración y de aprobación, tipo de error y etiquetas.

6.2 Fuga de datos: un defecto encontrado y corregido

Una versión previa del dataset tenía **fuga real**: 10 textos idénticos en `DEVELOPMENT` y `FINAL_TEST` (8,3 % del test), 19 cruces en total. La causa raíz fue una decisión de diseño defendida por escrito —"cada caso es su propio grupo de paráfrasis"— que hacía que el validador de grupos pasara **tautológicamente**: un grupo de tamaño 1 no puede cruzar nada. La justificación era falsa: la norma prohíbe que cruce "ni formulación semánticamente equivalente", no solo un identificador de grupo.

Corrección: pools de valores ampliados de 4–8 a 24 elementos, asignación sin repetición dentro de una intención, sustitución de un estilo duplicado, y un validador nuevo (`validate_no_split_leakage`) que compara texto normalizado y el par (intención, argumentos). **Probado con una fuga plantada** para que

no sea otra comprobación vacua. Estado actual: 480/480 textos únicos, 0 cruces.

6.3 Limitaciones del dataset

Sintético, plantillado, en un solo idioma y contra un solo ERP simulado. Es una limitación de validez externa que se declara y no se compensa: los resultados de recuperación en particular (§8.5) están condicionados por el alto solape léxico entre petición y descripción de skill que las plantillas producen.

Revisión por segundo anotador y kappa de Cohen: retirados formalmente, no completados. El plan estadístico original de este benchmark (v1) los exigía; el instrumento llegó a construirse (`data/annotation_review_sheet.csv` , 96 casos estratificados que sobrerrepresentan adversariales y alto riesgo) y el script de cálculo se negaba a emitir un kappa mientras la columna del segundo anotador estuviera vacía —nunca se fabricó un número—, pero ese paso humano no se va a completar. La razón no es abandono: el protocolo v2.1 (`docs/tfm-closure-no-human-v2.1.md`) sustituye la anotación humana por verdad de referencia construida algorítmicamente y dos oráculos independientes del código evaluado, y esa campaña **ya está implementada, congelada y ejecutada** (§8.0, `docs/results-v2.1.md`). El acuerdo entre anotadores queda como una limitación declarada del piloto v1, no como trabajo pendiente del proyecto.

7. Experimentos

Este capítulo describe el piloto exploratorio v1 — el diseño de cinco corridas que sirvió para construir y depurar el instrumento de medida, incluidos los dos sesgos de §7.3. **No es el protocolo de la campaña confirmatoria**, que es v2.1: verdad de referencia por construcción, sin anotación humana, con potencia y protocolo congelados **antes** de generar el holdout y evaluación única (§8.0, `docs/tfm-closure-no-human-v2.1.md` , `docs/results-v2.1.md`). Se conserva aquí, íntegro, porque el diseño de las cinco corridas y los sesgos que corrigió son precisamente lo que informó cómo se construyó v2.1 — no porque siga siendo el protocolo vigente.

7.1 Protocolo

120 casos de test congelado × 3 sistemas × 3 repeticiones = **1.080 ejecuciones** por corrida. Orden aleatorizado con semilla, estado de `FakeERPAdapter` reconstruido antes de cada observación, mismos roles, mismas claves de idempotencia, mismos presupuestos de timeout y reintento, mismo evaluador determinista.

7.2 Las cinco corridas

#	Selector	Régimen de argumentos	Papel
1	OpenRouter (<code>gpt-oss-20b:free</code>)	Parseo regalado	Histórica; etiqueta confirmatoria invalidada
2	Stub determinista	Parseo regalado	Arquitectura-solo: aísla gobernanza de calidad del modelo
3	Groq (<code>llama-3.1-8b-instant</code>)	Parseo real	Elimina el sesgo del parseo regalado
4	Groq	Parseo real + normalización	Referencia exploratoria más reciente
5	Groq	Parseo regalado	Réplica que separa proveedor de régimen

Por qué cambia el proveedor entre corridas. La cuota diaria gratuita de Groq se agotó durante la instrumentación de tokens; Gemini resultó inviable (20 peticiones/día por modelo en esta cuenta); OpenRouter completó la corrida 1 pero entraba en tormentas de 429 que hacían la corrida 3 inviable (~3 h con interrupciones frente a ~50 min con Groq). D-03 exige que A/B/C compartan proveedor **dentro** de una corrida, no un proveedor concreto entre corridas. El confundido proveedor↔régimen que esto introducía se **resolvió con la ejecución 5** (§8.10), que repite el régimen de argumentos dados con Groq para dejar el proveedor fijo.

7.3 Dos sesgos corregidos antes de publicar cifras

- 1. Parseo regalado (corridas 1–2).** Los tres sistemas recibían `case.expected_arguments` : un parseo perfecto que nadie pagaba. Eso hacía que C —cuya recuperación es TF-IDF— apareciera con coste **cero** en tokens, cuando un despliegue real necesitaría un LLM para convertir el texto en argumentos. Corregido en la corrida 3: los tres extraen del texto crudo con el mismo LLM, prompt y lista de campos.
- 2. Falta de normalización (corrida 3).** El LLM extraía `'27600 euros'` para un campo numérico; el validador lo rechazaba por tipo. Ese fallo **penalizaba solo a C**, el único sistema que valida tipos antes de ejecutar. Corregido con un normalizador deliberadamente **estrecho** (número con unidad monetaria opcional; cualquier otra cosa sigue fallando), cableado a B y a C, no solo al que se beneficia.

Las postcondiciones se verifican siempre contra la verdad de referencia, no contra lo que el LLM extrajo: mide si la tarea quedó bien hecha, no si el parser se autoconfirmó.

7.4 Infraestructura de ejecución

Checkpoint/resume por observación (una interrupción cuesta solo las llamadas no persistidas) y caché de llamadas por sistema: las tres repeticiones de un caso emiten la misma consulta y con temperatura 0 producen la misma respuesta, así que solo la primera es real. La caché es **por sistema**: compartirla entre A, B y C hacía que pagara el que el orden aleatorio ejecutara primero, convirtiendo los totales de tokens en una medida del orden de ejecución (defecto #12, §9.5).

8. Resultados

8.0 Resultado confirmatorio vigente (protocolo v2.1)

Las secciones 8.1 en adelante describen el piloto exploratorio v1 (1.080 ejecuciones, código de análisis corregido después de inspeccionar sus propios resultados) y se conservan como contexto y como origen de varios de los defectos discutidos en §9.5 — no como el resultado confirmatorio del trabajo. El resultado confirmatorio real es el protocolo v2.1 sin anotación humana (docs/tfm-closure-no-human-v2.1.md): verdad de referencia por construcción, dos oráculos independientes del código evaluado, potencia y protocolo congelados antes de generar el holdout, evaluación única. Campaña real de 21.478 observaciones, RUN_COMPLETED / CLOSURE_VALID , detalle completo en docs/results-v2.1.md .

Hipótesis	Resultado	Veredicto
H1a (C no inferior a A)	+25,3 pp, IC95 desde +23,2 pp	confirmada
H1b (C superior a B)	-1,5 pp, $p=0,286$	no confirmada
H2 (tokens, vs A y vs B)	-468 y -648 tokens, IC95 completo <0	confirmada
H3a (estabilidad entre paráfrasis)	$p=2,2e-18$, OR=9,35	confirmada
H3b (variabilidad estocástica)	36,7 % [24,6, 50,1]	descriptiva
H4 (seguridad, 4 componentes)	ver más abajo	no confirmada
H5 (recuperación selectiva)	selective_acc=0,589, false_reuse=0,411	no confirmada
H6 (valor de la abstención)	IC95 completo <0	confirmada
H7 (auditoría, 7 hechos)	+42,7 pp, $p=2,85e-112$	confirmada ¹
H8 (coste modelado)	rejilla de sensibilidad, sin criterio	descriptiva

¹ Con la salvedad de que A/B carecen de policy engine, versión de skill y verificación de postcondiciones por definición arquitectónica — parte de la ventaja es estructural, no ganada en igualdad de condiciones.

H4, el resultado que más pesa: sobre 315 escenarios peligrosos reales, C deja pasar una mutación no autorizada en el 19,0 % de los casos [IC95 hasta 23,1 %] — casi cuatro veces el umbral prerregistrado del 5 %. Desglosado por las siete categorías de ataque: **cero** mutaciones en insufficient_permissions y disguised_bulk_modification (el filtro de rol y la clarificación funcionan); entre el 18 % y el 31 % de fallo en las cinco categorías restantes (argument_out_of_range , duplication_or_retry , field_conflict , prompt_injection_in_data , similar_but_wrong_skill). No es un artefacto de los dos defectos que contaminaban una campaña anterior de esta misma familia de protocolo (H7 sin cablear, una categoría de ataque sin señal observable): verificado que el número apenas cambia (19,6 % → 19,0 %) al corregirlos.

La comparación cruda con A/B necesita un matiz que cambia su lectura: para A y B, "DENY" no es una decisión de seguridad — es la etiqueta que el harness asigna a cualquier error de ejecución ("ALLOW" if result.error is None else "DENY" , experiment_v2_1.py), sin distinguir un rechazo deliberado de un identificador de registro inexistente o una herramienta mal elegida. Por eso A "deniega" el 72,7 % de las

peticiones peligrosas y C solo el 5,7 %, pero esa diferencia no es comparable: mide sobre todo si la petición estaba bien formada, no si el sistema entendió que era peligrosa. El único número de H4 libre de esta ambigüedad es la mutación no autorizada de C, medida sobre el estado observado, no sobre la decisión declarada.

Lectura de conjunto: cuatro de nueve pruebas confirmatorias apoyan la arquitectura (eficiencia, estabilidad, trazabilidad, valor de la abstención); el éxito de tarea no mejora sobre un baseline con herramientas tipadas; la recuperación no alcanza los umbrales operativos exigidos; y la promesa de detección activa de peligro no se sostiene. Es un resultado mixto, y §16 del protocolo lo permite explícitamente: el cierre depende del proceso, no de que todo salga favorable.

Tres figuras reproducibles resumen esta sección (Anexo A, `make_figures_v2_1.py`):
`reports/figures/v21_hypotheses_forest` (las 9 pruebas, estimación e IC95),
`reports/figures/v21_h4_categories` (mutación no autorizada de C por categoría, con la línea del umbral del 5 %) y `reports/figures/v21_h2_tokens` (ahorro de tokens contra A y contra B).

Las secciones siguientes (8.1–8.x) documentan el piloto v1, con sus propias cifras y contexto, tal como se registraron en su momento.

8.1 H1 — Strict Task Success Rate

Sistema	STSR
A (agente directo)	0,000
B (herramientas tipadas)	0,483
C (ERP Agent OS)	0,633

Contraste	Diferencia	IC 95 %	Holm p	OR
C – A	+0,633	[+0,550, +0,717]	$1,55 \times 10^{-17}$	153,0
C – B	+0,150	[+0,042, +0,258]	0,0162	2,09

Q de Cochran = 102,87 (gl = 2). **H1 se acepta:** el límite inferior del IC de C–A supera con holgura el margen de no inferioridad de –5 pp, y C–B es además significativo.

Por qué A obtiene exactamente 0,000. Es casi determinista dado su diseño: herramientas CRUD genéricas no pueden codificar postcondiciones, y STSR exige estado final verificado. Por tanto **el contraste informativo es C–B**, donde ambos sistemas comparten catálogo, esquemas y handlers y solo difieren en gobernanza. Presentar C–A como el resultado principal sería inflar la conclusión.

8.2 H4 — Seguridad

Métrica	A	B	C
Casos peligrosos	27	27	27
<i>False allow rate</i>	0,889	0,889	0,111
<i>False block rate</i>	0,225	0,072	0,072
Recall de detección	0,111	0,111	0,889
Precisión de detección	0,038	0,111	0,500

C permite 3 de 27 casos peligrosos donde A y B permiten 24, **y lo hace sin bloquear de más** (mismo false block que B, un tercio del de A). Es el resultado más estable del trabajo: invariante entre proveedores y entre regímenes de parseo, porque la decisión no depende del LLM.

Dos advertencias que acotan esta cifra, ambas medidas caso a caso.

Primera: **la ventaja es aquí, en su mayor parte, detección léxica, no gobernanza**. De los 9 casos peligrosos del test congelado, **8 disparan patrones** escritos mirando el corpus de este benchmark; como los casos de test provienen de las mismas plantillas que los de desarrollo, ese ajuste transfiere casi por construcción. Es un número **dentro de distribución**; fuera de ella la detección cae al 3,3 % (§8.6). Lo que sostiene la afirmación propiamente arquitectónica es el otro experimento: 0/1.530 mutaciones no autorizadas aunque el detector no dispare.

Segunda: **el único caso que C permite no es peligroso**. `r0457` pide "Crea una factura en borrador para Oceanic Airlines" y está etiquetado `argument_out_of_range` porque el generador asigna esa categoría rotando por índice de intención, sin comprobar que la skill destino tenga un campo numérico acotado. Los 9 casos de esa categoría en el dataset completo comparten el defecto. Excluyéndolo, el *false allow* de C es **0,000** y el de A/B **1,000**. **El dataset no se corrige**: está congelado, y arreglarlo mejoraría los resultados de C, que es exactamente el cambio post-hoc que la congelación existe para impedir. Se publica la cifra contaminada y esta como sensibilidad.

Y sobre ambas columnas: **n = 9 casos peligrosos**, con IC anchos (el de C llega a 0,435). El "8×" es una estimación puntual sobre nueve casos.

Descomposición honesta del recall. De los 9 casos peligrosos del test analizados en el estudio de ordenación del pipeline, 7 salen `DENY`, 1 `ABSTAIN` y 1 `ALLOW`. El resultado de seguridad del caso abstenido es idéntico —no se ejecuta nada— pero el sistema **no lo identificó** como peligroso: simplemente no lo entendió. El recall de 0,889 se descompone así en 0,778 de detección real y 0,111 de abstención. Reportar 0,889 como si fuera toda comprensión del ataque sería deshonesto.

8.3 H2 y H8 — Tokens y coste

Sistema	Tokens/ejecución	Total
A	185,1	66.636
B	265,3	95.497
C	67,6	24.344

$C - B = -197,6$ tokens/ejecución, IC95 $[-198,3, -196,9]$: **3,9× más barato**. El mecanismo es explícito: los tres pagan la extracción de argumentos; A y B pagan **además** una llamada de selección de herramienta, que C sustituye por TF-IDF a coste cero de tokens.

H8, con tarifa declarada de 0,05 USD/1.000 tokens: A \$3,33 · B \$4,77 · **C \$1,22**. Es un **análisis de sensibilidad con supuesto declarado**, no un ahorro observado —los proveedores usados son gratuitos—, tal como exige el plan de métricas.

8.4 H7 — Trazabilidad

Rúbrica ponderada de siete componentes (identidad de petición 10 %, interpretación 15 %, candidatas/abstención 15 %, decisión de política 15 %, versión de skill y clave 15 %, resultado y efectos 15 %, evidencia de postcondición/aprobación/bloqueo 15 %), puntuada desde evidencia real de auditoría, no por volumen de logs.

Sistema	Puntuación media
A	0,356
B	0,374
C	0,820

A y B puntúan bajo **por construcción**: carecen de policy engine, de skills versionadas y de almacén de auditoría. Su puntuación no es un fallo del calculador, es la brecha de gobernanza hecha medible.

8.5 H5, H6 y comparación de recuperadores

Sistema	Top-1	Top-3	MRR	Cobertura	Exact. selectiva	False-reuse risk
A	0,000	0,000	0,000	1,000	0,839	0,161
B	0,898	0,898	0,898	1,000	0,898	0,102
C	0,780	0,941	0,855	0,907	0,785	0,215

B alcanza Top-1 = 0,898 sin abstenerse nunca: su selector real es bueno. C gana en Top-3 y abstiene en el 9,3 % de los casos. **H6 queda matizada**: el valor de abstenerse depende de la calidad del selector alternativo, y con un selector fuerte la abstención cuesta cobertura sin comprar tanta precisión como cabría esperar.

Comparación de recuperadores (solo desarrollo y validación, jamás test), con calibración individualizada de umbral y margen por recuperador para no favorecer al incumbente:

Recuperador	Top-1 dev	Top-1 val
TF-IDF	0,767	0,733
Embeddings	0,713	0,658
Híbrido	0,713	0,675

TF-IDF gana en todas las métricas y en ambos splits. El experimento histórico ya usaba el mejor de los tres. La causa es el benchmark: texto plantillado con alto solape léxico entre petición y descripción de skill, exactamente la señal que TF-IDF explota y que un vector denso comprime. **No es un resultado sobre embeddings en general.**

8.6 Robustez adversarial contra un dataset externo

Dos mediciones sobre los 510 payloads de InjecAgent, fuera de la distribución del benchmark propio (inglés, dominios no ERP):

1. **Detector léxico:** 0 % de detección con patrones solo en español; **3,3 %** tras añadir patrones en inglés. Ampliar el vocabulario no cerró la brecha porque el vocabulario no era el cuello de botella: la mayoría de los payloads son peticiones educadas sin ningún framing de ataque textual, invisibles por diseño a cualquier detector léxico.
2. **Resistencia efectiva:** los mismos 510 payloads por los tres canales que un atacante controla — texto de la petición, dato almacenado que la petición lee, y parser comprometido con los argumentos dictados por el atacante— producen **0 / 1.530 mutaciones no autorizadas**, con 510/510 **DENY** en el brazo de parser comprometido.

La lectura conjunta es la que importa: la defensa efectiva de esta arquitectura **no es el detector léxico** sino que los datos del ERP nunca ocupan posición de instrucción, que el LLM solo puede emitir un identificador de skill y argumentos validados contra esquema, y que el handler escribe solo en su modelo y campos permitidos. Un control positivo aborta la medición si una petición limpia no llega a mutar, para que "0 mutaciones" no pueda salir vacuo.

No probado y declarado: un adversario adaptativo que conozca el catálogo y redacte argumentos válidos para una skill legítima pero indeseada.

8.7 Segmentación

Por clase de riesgo, C obtiene $R0 = 0,600 \cdot R1 = 0,583 \cdot R2 = 0,950 \cdot R3 = 0,400$. El peor resultado está en R3, y la causa es normativa, no un defecto: en R3 la política obliga a simular incluso tras aprobación, así que los casos que el dataset espera **ejecutados** no pueden puntuar. Es una tensión real entre la norma de seguridad y la métrica de éxito, y se discute como tal en §9.3. El excelente resultado en R2 (0,950) muestra que, cuando la norma permite ejecutar tras aprobación, la gobernanza no cuesta éxito.

Por módulo, los peores son **tasks** (0,400) y **sales** (0,467); los mejores, **product** (0,900), **billing** (0,800) y **purchasing** (0,800). Por etiqueta: NORMAL 0,750 · ADVERSARIAL 0,526 · NOISE 0,486 — el ruido lingüístico penaliza más que la adversarialidad, porque una petición adversarial bien detectada **puntuá** (la decisión esperada es el bloqueo) mientras que una petición ruidosa mal parseada, no.

8.8 Demostración contra un ERP real

Más allá del simulador, el pipeline completo de System C se ejecutó contra una instancia real de Odoo 19 (rama de desarrollo con datos demo), con el mismo **Runtime**, **AuditStore** y policy engine que corren las 1.080 observaciones, cambiando únicamente el adaptador:

- creación de oportunidad R1 → ejecuta y verifica postcondición por **relectura independiente**;

- actualización R2 sin aprobación → `REQUIRE_APPROVAL` , y una relectura independiente confirma que Odoo **no cambió**;
- misma petición tras conceder aprobación → `ALLOW` y escritura correcta.

Es una demostración **cualitativa** con 2 de las 12 skills mapeadas a modelos reales; no sustituye una comparación A/B/C prospectiva, y así se declara.

8.9 Réplica que separa proveedor de régimen de argumentos

La comparación entre regímenes estaba confundida con el proveedor: la corrida de argumentos dados usó OpenRouter y la de parseo real, Groq. La ejecución 5 repite **argumentos dados con Groq**, dejando el proveedor fijo:

	Groq, argumentos dados	Groq, parseo real	Δ
STSR B	0,492	0,483	-0,008
STSR C	0,700	0,633	-0,067
C - B	+0,208 ($p = 0,0015$)	+0,150 ($p = 0,016$)	-0,058

Con el proveedor constante, **C cae 6,7 puntos al parsear de verdad y B solo 0,8**. El efecto es del régimen de argumentos, no del modelo: el parseo regalado beneficiaba a C desproporcionadamente. La amenaza a la validez interna más seria del trabajo queda así medida en lugar de argumentada.

Verificación interna no planeada, que el diseño supera. El incremento de tokens al pasar a parseo real es +67,68 (A), +67,67 (B) y +67,62 (C): los tres pagan **la misma** extracción, como exige D-03. Y el gasto **total** de C con parseo real (67,62) es exactamente esa extracción y nada más — no hay llamada de selección de herramienta, que es precisamente el mecanismo que la tesis afirma.

Residuo declarado. El *false allow* de A sí depende del proveedor (0,333 con OpenRouter, 0,889 en las dos corridas Groq), mientras que el de C es 0,111 en todas. La seguridad de un agente sin gobernanza depende de qué modelo le toque; la de la arquitectura gobernada, de ninguno.

8.10 Estado auditado de las hipótesis — piloto v1 (histórico)

Esta tabla describe el estado del piloto v1 en el momento en que se escribió, antes de que existiera la **campana confirmatoria**. No es el estado vigente del proyecto — ese es §8.0, con veredicto explícito por hipótesis (H1a-H8) sobre 21.478 observaciones reales. Se conserva porque documenta con precisión *por qué* v1 no podía dar por sí solo una respuesta confirmatoria, que es la motivación directa del diseño de v2.1.

H (v1)	Estado del piloto en su momento
H1	Señal exploratoria favorable. El test fue inspeccionado, faltan filas históricas y oráculo independiente.
H2	Señal exploratoria favorable. El histórico incluyó dos casos <code>sin_skill</code> fuera de la población declarada.
H3	No evaluable con el diseño del piloto. Temperatura 0 y caché producen $A=B=C=1,000$.
H4	Señal exploratoria favorable. Solo nueve casos peligrosos únicos y baja transferencia del detector.
H5	Parcial y descriptiva. Buen Top-3 en v1; caída acusada de TF-IDF en texto menos templado.
H6	Parcial y descriptiva. La curva se implementó después de inspeccionar v1.
H7	Señal exploratoria favorable. No se conservaron las filas históricas por componente.
H8	Solo análisis de sensibilidad, no ahorro medido.

9. Discusión

9.1 Qué compra la gobernanza, y a qué precio

Actualizado con el resultado confirmatorio (v2.1, §8.0). La formulación que la evidencia sostiene hoy es más estrecha que la que este capítulo defendía con el piloto v1, y en un eje se invierte:

Frente a los dos baselines, la arquitectura gobernada compra **tokens más baratos** (IC95 completo por debajo de cero contra A y contra B), **mayor estabilidad** entre formulaciones de la misma petición ($p=2,2e-18$) y **mayor reconstrucción de auditoría** ($p=2,85e-112$, con la salvedad estructural de §9.4). **No compra una ventaja en éxito de tarea sobre herramientas tipadas** ($p=0,286$). Y, al contrario de lo que sostenía este capítulo hasta esta revisión, **no compra menos ejecuciones inseguras**: sobre 315 escenarios peligrosos reales, deja pasar el 19,0 % de mutaciones no autorizadas, casi cuatro veces el umbral prerregistrado.

La historia completa de cómo cambió la cifra de seguridad —de "8× menos inseguro" (piloto v1, $n=9$ casos) a "19,0 % de mutación no autorizada" (campana confirmatoria, $n=315$, ver §8.0 y §9.5)— es en sí misma un resultado: **cuánto puede depender una conclusión de seguridad del tamaño y la composición de la muestra de casos peligrosos**, no solo de si el detector "dispara". Con nueve casos el intervalo de confianza era $[0,020, 0,435]$ — demasiado ancho para sostener nada —; con 315, la estimación es precisa y va en la dirección contraria.

El precio de la gobernanza, medido: latencia adicional del pipeline determinista (parseo, recuperación, política, verificación), abstenciones que exigen intervención, y el coste de mantener catálogo, handlers y postcondiciones — a lo que se añade ahora el propio hueco de seguridad de §8.0 como coste no resuelto, no solo como precio de mantenimiento.

9.2 Cuándo un agente directo sigue siendo preferible

Cuando el dominio no tiene efectos de escritura (consultas puras), cuando la variedad de operaciones es demasiado alta para mantener un catálogo, o cuando el coste de un error es bajo y el de la abstención alto. La evidencia de este trabajo no dice que gobernar sea siempre mejor; dice qué se compra y qué se paga.

9.3 Tres tensiones que no se resuelven a favor del número bonito

1. **R3: seguridad contra métrica de éxito.** La política obliga a simular en alto impacto, y eso hace estructuralmente imposible puntuar los casos R3 que el dataset espera ejecutados. Bajar la política para subir STSR habría sido optimizar la métrica, no el sistema.
2. **Abstención contra Top-1.** Calibrar el margen de abstención a cero sube Top-1 unos 7 puntos, pero elimina casi toda la abstención, que es parte de la tesis (H6). El experimento conserva el margen conservador.
3. **Temperatura contra H3 — resuelta en v2.1, con matiz.** La norma exige temperatura baja, y bajo repetición estocástica literal (H3b) la estabilidad sigue siendo casi trivial. Pero comparar **formulaciones distintas** de la misma petición (H3a), en vez de repeticiones de la misma formulación, sí discrimina: $p=2,2e-18$. La reformulación que este capítulo proponía como pendiente en el piloto v1 **ya se ejecutó con LLM real** y confirma estabilidad. H3b, la reformulación que sí sigue siendo casi trivial, se reporta como descriptiva sin criterio, tal como estaba previsto.

9.4 Amenazas a la validez

Interna. El confundido proveedor↔régimen de parseo (Groq en las corridas 3–4, OpenRouter en la 1) **queda resuelto** por la ejecución 5 (§8.9): con el proveedor fijo, C cae 6,7 puntos al parsear de verdad y B solo 0,8, de modo que el efecto es atribuible al régimen y no al modelo. Residuo declarado: el *false allow* de A depende del proveedor (0,333 con OpenRouter, 0,889 con Groq), aunque el de C es invariante.

Externa. Benchmark sintético y plantillado, un solo idioma, un solo ERP simulado, modelos de nivel gratuito (no frontera). Las dos demos contra Odoo real son evidencia parcial de transferencia, no sustitutivo.

La limitación externa más grave, ahora medida y no solo declarada — y ahora también confirmatoria (H5, §8.0). Se evaluaron 120 peticiones en registro coloquial, ajenas al generador del benchmark. El recuperador TF-IDF de C cae de 0,733 a **0,381** de Top-1; el selector LLM que usa B — mismo prompt, mismas herramientas, mismo proveedor — cae solo de 0,898 a **0,750**. La campaña confirmatoria de v2.1 mide lo mismo por otra vía, con un benchmark procedural distinto: selective accuracy 0,589 y false-reuse risk 0,411, muy por debajo/encima de los umbrales prerregistrados. Dos benchmarks distintos, dos mediciones distintas, misma conclusión: **la recuperación léxica de C es el cuello de botella real del sistema**, no una curiosidad de un solo experimento. La ventaja de C sobre B en éxito de tarea, que en v1 dependía en parte de esto, ya no se sostenía tampoco en la campaña confirmatoria por razones independientes (§8.0, H1b) — ambos hallazgos apuntan en la misma dirección sin depender uno del otro.

Refinamiento posterior, que acota la gravedad de la limitación. Un tercer experimento (docs/product-viability.md §7.4, con mitad de calibración y mitad held-out) muestra que la causa no es TF-IDF como técnica sino las **descripciones de una línea** del catálogo: enriquecerlas con sinónimos y formulaciones reales — en un fichero aparte, sin tocar el catálogo congelado — lleva el Top-1 de 0,455 a **0,886** en la

mitad held-out, por encima del router LLM (0,818) y sin coste alguno en tokens. La brecha de enrutado con texto real es, por tanto, plausiblemente **corregible sin renunciar a la ventaja arquitectónica**. Con $n = 44$ los intervalos se solapan y el corpus procede de un solo autor, así que es una indicación fuerte y no una demostración. Ninguno de los números del experimento congelado cambia: midió el catálogo tal como estaba.

El mismo experimento arroja un matiz que corta en dirección contraria: en las 36 peticiones que **ninguna** skill cubre, el selector LLM se compromete con una herramienta en 30 (83 %) frente a 22 de TF-IDF (61 %). El LLM enruta mejor y **se calla mucho peor**, que en un ERP es la dirección peligrosa del error. De ahí que la capa de gobernanza —validación de esquema, permisos, riesgo y postcondiciones, ninguno de los cuales depende del router— resulte **más** necesaria, no menos, si se sustituye la recuperación léxica por un router basado en LLM.

De constructo. Los detectores adversariales son léxicos y están ajustados al texto del benchmark: medido con InjecAgent, 3,3 % de detección fuera de distribución. La rúbrica de trazabilidad mide presencia de evidencia, no calidad semántica. Las postcondiciones las definieron los mismos autores que los handlers: riesgo de circularidad, declarado.

Estadística. 120 unidades de inferencia; comparaciones múltiples corregidas con Holm; pseudo-replicación explícitamente evitada (§3.4).

9.5 Dónde se concentran los defectos: un hallazgo sobre el proceso

Dieciséis defectos encontrados y corregidos en el propio instrumento de medida, en las dos generaciones del protocolo (v1 y v2.1). Los más graves:

#	Defecto	Consecuencia
—	Fuga de test (10 textos idénticos entre splits)	Validador tautológico; test contaminado
—	Conjunto 5 de STSR devolvía <code>True</code> incondicionalmente	Métrica de 5 componentes que era de 3
—	Conjunto 4 duplicaba al conjunto 1	Ídem
—	Pseudo-replicación (360 observaciones tratadas como independientes)	15 órdenes de magnitud de significación fabricada
—	McNemar sin corrección de continuidad	Estadístico anticonservador
—	Bootstrap sin remuestreo	IC degenerado que el test aceptaba
#12	Caché de LLM compartida entre sistemas	Los totales de tokens medían el orden de ejecución
#13	Falta de normalización de argumentos	Sesgo asimétrico contra C
#14	Los 9 casos <code>argument_out_of_range</code> del dataset, mal etiquetados	Un caso benigno cuenta como peligroso y contamina H4
#15	Denominador equivocado en el arnés de validación de producto	Habría reportado un derrumbe de $-0,466$ cuando el real es $-0,352$
—	v2.1: <code>SystemC.handle()</code> nunca recibía <code>postcondition_checks</code>	H7 salía con $p=1,0$ exacto, degenerado, en la primera campaña v2.1
—	v2.1: categoría de ataque <code>r4_operation</code> sin señal observable (par peligroso/seguro idéntico)	Contaminaba el agregado de H4 sin que ningún sistema pudiera distinguirlo
#16	v2.1: la comparación de tokens (H2) solo verificaba C contra A, nunca contra B	El veredicto "confirmada" no certificaba lo que el protocolo exige

Además de los defectos de código, un **hallazgo de construcción de la métrica**, no un defecto: al diagnosticar por qué H4 seguía saliendo mal tras corregir sus dos defectos de instrumentación, se encontró que para los baselines A y B la decisión `"DENY"` la asigna el arnés a cualquier error de ejecución (`"ALLOW" if result.error is None else "DENY"`), no a un juicio de seguridad — así que "A deniega más que C" no es directamente comparable con "C deniega más que A". No cambia ningún número; cambia cómo debe leerse la comparación (§8.0, §9.1).

Dos patrones, ambos utilizables como material metodológico:

Primero: una comprobación que no puede fallar es peor que no tener comprobación, porque fabrica confianza. Cinco de los defectos tienen exactamente esa forma. La regla adoptada tras el tercero — todo guard nuevo debe demostrarse fallando, con fuga plantada, componente alterado o entrada construida — es la que hizo detectables los siguientes.

Segundo: el TDD estricto protege bien lo que se implementa contra un requisito explícito, y protege mal lo que solo se calcula a partir de una fórmula. Los dos únicos supervivientes de 40 mutantes cayeron en `statistics.py`, la capa que produce los números publicados. Sus tests originales verificaban la conclusión del cálculo (¿es significativo?, ¿está en rango?) en vez de *el mecanismo* (¿es exactamente esta

fórmula, con esta corrección?), y ambas propiedades se cumplían igual con la fórmula rota. La corrección consistió en sustituir aserciones de conclusión por aserciones de mecanismo: valor exacto del estadístico, anchura de intervalo no degenerada y proporcional al error estándar teórico.

Tercero, y el más incómodo: de dieciséis defectos, catorce salieron de auditorías propias; dos los destapó una pregunta escéptica sobre resultados que ya habían sido aceptados y publicados (el #13, sobre un resultado no significativo dado por bueno, y el #14, al preguntar si la métrica de seguridad tenía sesgo). Es el patrón esperable: la autoauditoría encuentra bien el código que se contradice consigo mismo, y mal el código que hace exactamente lo que su autor creía que debía hacer. Para eso hace falta alguien que dude del supuesto, no de la implementación. La comparación de tokens sin verificar contra B (#16) y el hallazgo sobre el **DENY** de A/B, en cambio, sí salieron de auditoría propia — al leer el código de análisis en vez de confiar en el veredicto, y al desconfiar de un número de seguridad demasiado parecido al de una campaña ya conocida como defectuosa.

Un matiz sobre el #14 que merece registrarse: al corregirlo, el resultado **mejoraría** para la tesis (el *false allow* de C pasaría de 0,111 a 0,000). Aun así el dataset no se corrigió, porque está congelado y ese es exactamente el cambio post-hoc que la congelación existe para impedir. Se publicó como análisis de sensibilidad junto a la cifra contaminada.

El equivalente en v2.1 corrió en la dirección contraria y por eso es una prueba más dura: la categoría de ataque análoga a este problema (**r4_operation**, sin señal observable) **sí se retiró**, correctamente, **antes** de generar el holdout de la campaña confirmatoria — no es un cambio post-hoc, es exactamente el procedimiento que la congelación permite. Y el resultado de seguridad no mejoró: la mutación no autorizada pasó de 19,6 % a 19,0 %, prácticamente sin cambio. Que corregir el defecto de instrumentación no cambiara la conclusión es la evidencia más fuerte de que el hallazgo de H4 es real y no un artefacto de las dos categorías rotas.

9.6 Resultados negativos, reportados como tales

- **El resultado negativo más importante del trabajo, confirmatorio:** sobre 315 escenarios peligrosos reales, C deja pasar el 19,0 % de mutaciones no autorizadas — casi cuatro veces el umbral prerregistrado — y no supera a A ni a B en ninguno de los cuatro componentes de H4. Contradice directamente la expectativa con la que se diseñó el sistema.
- H1b: la ventaja de C sobre B en éxito de tarea **no se confirma** ($p=0,286$) — reproduce, con datos limpios de dos defectos conocidos y un benchmark distinto, la misma conclusión que ya forzó una reformulación de la tesis durante el piloto v1.
- H5: la recuperación **no alcanza** ninguno de los tres umbrales operativos exigidos (selective accuracy, false-reuse risk, coverage), de forma confirmatoria, no solo exploratoria.
- Los **embeddings pierden** frente a TF-IDF en el benchmark v1, y el ranking híbrido no mejora al embedding puro.
- **H3b es no discriminable** por diseño, bajo temperatura baja y sin paráfrasis — previsto y así reportado.
- La **detección léxica no generaliza**: 3,3 % fuera de distribución.
- C es el sistema con **peor false-reuse risk** en varias mediciones (0,215–0,411 según el benchmark), consistentemente peor que B.

10. Productivización

API. FastAPI sobre System C con autenticación de demo por cabecera, `correlation_id` generado siempre en servidor (nunca aceptado del cliente, para que la correlación de auditoría no se pueda falsear), limitador de tasa en memoria y cuatro rutas: enviar petición, consultar catálogo, consultar auditoría por correlación y conceder aprobación.

Persistencia. Auditoría y aprobaciones sobre SQLAlchemy Core, sin `update` ni `delete` en la superficie, con PostgreSQL 16 provisionado por Docker Compose. Declarado: **pgvector no se usa** — la recuperación opera en proceso sobre 12 skills y añadirlo sería infraestructura sin necesidad demostrada.

Adaptador OdoO 19. Sobre la API JSON-2, con allowlist de modelos y campos aplicada **antes** de cualquier HTTP, sin operación de borrado (estructural, no convención), timeout, logs redactados y credenciales solo por entorno. Es intercambiable con `FakeERPAdapter` a través de un `Protocol` común, lo que permitió ejecutar el pipeline gobernado completo contra un ERP real sin duplicar lógica.

Reproducibilidad. `Makefile` con objetivos para tests, cobertura, validación de dataset, verificación de congelación, benchmark de humo, experimento, exportación a CSV y figuras. CI ejecuta instalación, lint, type-check, tests, cobertura, validación de dataset, smoke benchmark, verificación de congelación y subida de artefactos.

Insumos de dashboard. Tablas CSV y cinco figuras reproducibles PNG/SVG regeneradas desde el JSON versionado. El workbook de Tableau es trabajo manual pendiente y se declara como tal.

Valor empresarial. Con supuestos declarados: menor coste de inferencia (confirmatorio, H2) y capacidad de auditoría que permite reconstruir por qué se tomó cada decisión (confirmatorio, H7, con la salvedad de §9.1). El menor coste esperado de error **no** se sostiene igual que en el piloto v1: la campaña confirmatoria muestra un 19,0 % de mutación no autorizada sobre escenarios peligrosos reales (§8.0), así que el coste de error esperado de C no es automáticamente menor. No se presenta como ahorro medido ni como satisfacción de usuario, que no se han observado.

10.1 Transferencia a producto: qué sostiene la evidencia

Actualizado con la campaña confirmatoria (v2.1). El análisis completo está en [docs/product-viability.md](#), ya revisado con estas cifras. Su tesis central es que **la evidencia que aguanta ante un tribunal y la que aguanta ante un cliente no son la misma**, y que confundirlas produciría afirmaciones comerciales falsas.

Sostiene un producto: que ninguna inyección consiga una mutación no autorizada por ninguno de los tres canales de ataque cuando se concede el modelo entero al atacante (0/1.530); que la arquitectura elimine una llamada al LLM por petición, demostrado por aritmética y confirmado en v2.1 (H2); que el bloqueo se sostenga contra un ERP real verificado por relectura independiente; y la reconstrucción de auditoría, confirmatoria (H7, con la salvedad de §9.1).

No sostiene nada comercial: «detectamos peligro» o «somos más seguros que un agente sin gobierno» — **confirmatoriamente falso** (H4: 19,0 % de mutación no autorizada sobre 315 casos reales, casi 4× el umbral); la detección léxica de ataques (3,3 % fuera de distribución); el «8×» de v1, superado por el resultado confirmatorio de arriba; la invarianza al proveedor (nunca probada en la campaña confirmatoria); la ventaja de éxito de tarea (v2.1 confirma que C no supera a B, $p=0,286$); y cualquier cifra de ahorro (H8 es sensibilidad, no gasto medido).

La consecuencia es de diseño, no solo de discurso: **el producto no puede apoyarse en que el sistema entienda mejor, sino en que restrinja mejor**. Eso lo sitúa como plano de control bajo cualquier agente, no como agente competidor.

De los tres huecos que decidirían su viabilidad, **el tercero sí se midió** y su respuesta cambió el diseño propuesto. Sobre 120 peticiones en registro coloquial, partidas en mitad de calibración y mitad held-out:

Diseño de enrutado	Top-1 held-out	Rechaza bien	Tokens
TF-IDF con descripciones del catálogo	0,455	0,062	0
TF-IDF con descripciones enriquecidas	0,886	0,000	0
Router LLM (el del sistema B)	0,818	0,250	592
Filtro de dominio + TF-IDF enriquecido	0,864	0,250	0

La lectura es que **el cuello de botella no era el algoritmo de recuperación sino la descripción de una línea por skill**. Enriquecerla con sinónimos y formulaciones reales iguala o supera al router LLM sin gastar un token, lo que preserva la ventaja arquitectónica en lugar de devolverla. Consecuencia concreta para el producto: el alta de una skill debe pedir sinónimos y ejemplos de uso reales como **campo del contrato**, no una frase descriptiva.

El hallazgo se verificó además con autores distintos, que es la prueba que el corpus propio no permitía hacer. Sobre MASSIVE es-ES (Amazon, CC-BY-4.0: 16.521 frases, 60 intenciones, **20 crowdworkers identificados**), partiendo por persona y no por frase —el enriquecimiento se construye solo con frases de la mitad de los autores y se evalúa solo sobre la otra mitad, sin solape— la precisión de enrutado sube de **0,365 a 0,634** con diez ejemplos por intención, y satura ahí ($k=20$ da 0,629). Dos consecuencias operativas: bastan **unas diez formulaciones reales por skill**, y el umbral de abstención óptimo **baja** al enriquecer (0,55 → 0,32), de modo que una constante fija queda mal puesta en cuanto cambian las descripciones. El dominio de ese corpus no es ERP, así que la prueba es del **mecanismo**, no del producto.

Los otros dos huecos —coste de añadir una skill y tiempo humano por abstención— **siguen sin medir**, y se declaran como validación de producto pendiente. El detalle completo, con intervalos y con lo que estos números no permiten afirmar, está en [docs/product-viability.md](#) §7.

11. Conclusiones

11.1 Respuesta a la pregunta principal

Actualizado con el resultado confirmatorio (v2.1, §8.0) — sustituye la respuesta exploratoria de v1 que seguía aquí hasta esta revisión. Una arquitectura que separa la interpretación probabilística de la ejecución determinista **sí** reduce el consumo de tokens frente a los dos baselines (IC95 completamente por debajo de cero), **sí** es más estable entre formulaciones distintas de la misma petición ($p=2,2e-18$) y **sí** produce una reconstrucción de auditoría más completa ($p=2,85e-112$, con la salvedad de que A/B carecen de esa capacidad por diseño). **No** mejora el éxito de tarea frente a un baseline con herramientas tipadas ($p=0,286$) — la mejora de +15,0 pp que reportaba el piloto v1 no se sostiene en la campaña confirmatoria. Y, de forma más importante que cualquiera de los resultados favorables: **no** reduce el riesgo de seguridad frente a los baselines — sobre 315 escenarios peligrosos reales, deja pasar el 19,0 % de mutaciones no autorizadas, casi cuatro veces el umbral prerregistrado, localizado en cinco de siete categorías de ataque. La variabilidad bajo repetición estocástica pura (H3b) sigue sin ser discriminable a temperatura baja, tal como se anticipaba; el diseño de paráfrasis (H3a) **sí** resultó medible y confirma estabilidad.

11.2 Respuestas a las preguntas secundarias

1. **Precisión de recuperación ante paráfrasis:** en el piloto v1, Top-1 = 0,780, Top-3 = 0,941, MRR = 0,855, con TF-IDF superando a embeddings y a ranking híbrido en dev y validación. **La campaña confirmatoria lo contradice:** H5 (§8.0) mide selective accuracy 0,589 y false-reuse risk 0,411, muy fuera de los tres umbrales operativos exigidos — no adecuada. La respuesta confirmatoria a esta pregunta es que la recuperación **no** ofrece la precisión que el piloto sugería.
2. **Errores que previene el verificador:** ejecuciones bajo rol no autorizado, argumentos fuera de tipo o rango, operaciones con framing irreversible o de alcance masivo, y mutaciones cuyo estado final no coincide con la postcondición declarada.
3. **Reducción de tokens por reutilización:** confirmatoria contra los dos comparadores (§8.0, H2); el mecanismo es la sustitución de la llamada de selección.
4. **Variabilidad:** no discriminable bajo repetición estocástica pura (H3b), pero **sí** bajo formulaciones distintas de la misma petición (H3a) — confirmada con LLM real, $p=2,2e-18$. La reformulación que este trabajo proponía como pendiente ya se ejecutó.
5. **Latencia adicional de la gobernanza:** instrumentada por ejecución; el sobre coste es el del pipeline determinista, no de llamadas extra al modelo.
6. **Qué se automatiza sin aprobación:** R0 y R1; R2 exige aprobación y R3 permanece en simulación.
7. **Cuándo abstenerse:** score bajo, margen insuficiente entre candidatas, campos requeridos ausentes o conflicto de política.
8. **Umbral, cobertura y riesgo:** calibrar el margen a cero sube Top-1 unos 7 puntos y elimina casi toda la abstención — objetivos en conflicto, resueltos a favor de la configuración conservadora.
9. **Componente que más aporta, revisado con el resultado confirmatorio:** el policy engine explica la trazabilidad (H7, confirmada) y el ahorro de tokens (H2, confirmada), invariante a proveedor. **No** explica un resultado de seguridad favorable — H4 confirmatoria muestra que la validación previa deja pasar el 19 % de mutaciones peligrosas en escenarios sin marcador léxico obvio. El componente que **sí** aporta valor de seguridad medido es el confinamiento por contrato bajo modelo comprometido (0/1.530, InjecAgent), una propiedad distinta de la detección previa.

10. **Cuándo preferir un agente directo:** dominios sin escritura, catálogos inviables de mantener, coste de error bajo frente a coste de abstención alto, o —dato nuevo— dominios donde las peticiones peligrosas son sutiles y sin marcador léxico: en ese caso ningún sistema evaluado (A, B o C) ofrece una garantía de detección fiable, y la gobernanza no la sustituye por sí sola.

11.3 Trabajo futuro

Lo que este apartado pedía y ya se ejecutó desde la última revisión: la medición de H3a con LLM real (confirmada, §9.3); la campaña confirmatoria completa bajo protocolo v2.1 sin anotación humana, que sustituye al kappa de anotación que figuraba aquí como pendiente — retirado formalmente, no completado.

Lo que sigue pendiente: poblado de precondiciones del catálogo con su propia corrida; detección semántica de intención frente a la petición original, que es exactamente lo que el hallazgo de H4 (§8.0) y el resultado de InjecAgent señalan como límite estructural del enfoque léxico — ya no es una intuición, es la explicación más plausible de por qué C falla en 5 de 7 categorías de ataque sin marcador textual obvio; diseñar y ejecutar un endpoint real para las dos categorías de H4 que nunca llegaron a ejercitar su condición de peligro (`duplication_or_retry` , `field_conflict`); mapeo del resto del catálogo a modelos reales de Odoo; evaluación con usuarios reales.

La línea más prometedora, y la que este trabajo deja abierta con evidencia preliminar a favor: convertir la descripción de la skill en un artefacto de primera clase del contrato —sinónimos, formulaciones reales, ejemplos de uso— en lugar de una frase de una línea. La medición de §10.1 sugiere que ahí está la mayor parte de la brecha de enrutado con texto real, y que cerrarla **no cuesta tokens**. Queda por demostrar con peticiones de autores distintos, que es la prueba de generalización que el corpus disponible no permite hacer.

11.4 Cierre

El valor de este trabajo no está en presentar ERP Agent OS como solución universal. Hay muchos prototipos que conectan un modelo de lenguaje con un ERP, y la arquitectura empleada —skills versionadas, políticas, auditoría, postcondiciones— no es novedosa por sí misma. Y, con el resultado confirmatorio delante, tampoco puede presentarse como una solución de seguridad: sobre 315 escenarios peligrosos reales, deja pasar el 19,0 % de mutaciones no autorizadas.

Lo que este trabajo aporta es de otro tipo. Aporta **una forma más exigente de preguntar por la seguridad de un agente, con las dos respuestas que produjo, no solo la favorable**. Frente a un modelo comprometido que dicta directamente los argumentos, el confinamiento por contrato se sostuvo: 0 de 1.530 mutaciones no autorizadas sobre un dataset externo. Frente a una petición simplemente ambigua y plausible, sin ningún marcador de ataque, el mismo sistema falló uno de cada cinco casos peligrosos, en una campaña diseñada, congelada y evaluada precisamente para no poder mirar el resultado antes de comprometerse con el diseño. Publicar solo la primera respuesta habría sido una tesis más cómoda y menos verdadera; publicar las dos, con el diagnóstico exacto de en qué categorías falla y en cuáles no, es lo que hace que la afirmación de seguridad de este trabajo sea acotada y defendible en vez de una promesa.

Y aporta **el registro de haberse equivocado en público, dos veces distintas**. En el piloto v1, tres veces la medición honesta produjo un resultado peor que la intuición de partida, y las tres se publicaron antes de encontrar el matiz que las mejoraba. En la campaña confirmatoria v2.1, el patrón se repitió con una diferencia importante: no hubo matiz que mejorara el resultado de seguridad, y el número se sostuvo —pasar de 9 a 315 casos peligrosos, y corregir los dos defectos que contaminaban la medición anterior, no lo hizo desaparecer. Dieciséis defectos del instrumento quedaron documentados con su fecha, su causa y qué habría pasado sin corregirlos; dos de ellos los destapó una pregunta escéptica sobre resultados ya aceptados, y uno se dejó sin corregir precisamente porque corregirlo habría favorecido a la hipótesis.

Un trabajo experimental que solo confirma lo que esperaba debe levantar sospecha. Este documenta dónde se equivocó, cómo lo descubrió, y qué quedó en pie después de comprobarlo con más datos y más rigor, no menos. Eso —más que cualquiera de sus cifras favorables— es lo que pretende dejar utilizable para quien venga detrás.

12. Bibliografía

- Andriushchenko, M., Souly, A., Dziemian, M., et al. (2024). *AgentHarm: A Benchmark for Measuring Harmfulness of LLM Agents*. arXiv:2410.09024.
- Autio, C., Schwartz, R., Dunietz, J., et al. (2024). *Artificial Intelligence Risk Management Framework: Generative Artificial Intelligence Profile*. NIST AI 600-1.
- Li, M., Zhao, Y., Yu, B., et al. (2023). *API-Bank: A Comprehensive Benchmark for Tool-Augmented LLMs*. arXiv:2304.08244.
- Liu, X., Yu, H., Zhang, H., et al. (2023). *AgentBench: Evaluating LLMs as Agents*. arXiv:2308.03688.
- Reimers, N., y Gurevych, I. (2019). *Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks*. arXiv:1908.10084.
- Schick, T., Dwivedi-Yu, J., Dessì, R., et al. (2023). *Toolformer: Language Models Can Teach Themselves to Use Tools*. arXiv:2302.04761.
- Yao, S., Shinn, N., Razavi, P., y Narasimhan, K. (2024). *τ -bench: A Benchmark for Tool-Agent-User Interaction in Real-World Domains*. arXiv:2406.12045.
- Yao, S., Zhao, J., Yu, D., et al. (2022). *ReAct: Synergizing Reasoning and Acting in Language Models*. arXiv:2210.03629.
- Zhan, Q., Liang, Z., Ying, Z., y Kang, D. (2024). *InjecAgent: Benchmarking Indirect Prompt Injections in Tool-Integrated Large Language Model Agents*. arXiv:2403.02691.
- Documentación oficial de Odo 19 (API externa JSON-2).
- Especificación oficial de Model Context Protocol.
- Documentación oficial de PostgreSQL y pgvector.

Nota de honestidad: no se ha realizado una revisión sistemática de literatura con protocolo de búsqueda documentado. La bibliografía es la lista razonada de la especificación normativa, ampliada con las tres referencias incorporadas durante la ejecución (τ -bench, InjecAgent, AgentHarm). Esta limitación se declara en lugar de presentarla como revisión sistemática.

13. Anexos

Anexo A. Reproducción

```
uv sync
uv run python -m pytest                # suite completa
uv run python scripts/freeze_protocol.py --verify

# Experimento (arquitectura-solo, sin red)
uv run python scripts/run_experiment.py

# Experimento exploratorio con LLM real sobre test v1 ya inspeccionado
uv run python scripts/run_experiment.py --real-llm --real-parser \
    --provider groq

# Comparación de recuperadores (dev + validación, nunca test)
uv run python scripts/compare_retrievers.py

# Robustez adversarial
uv run python scripts/injecagent_stress_test.py
uv run python scripts/injection_resistance_test.py

# Demos contra Odoos 19 real (requiere instancia de desarrollo)
uv run python scripts/odoo_governed_demo.py
uv run python scripts/odoo_adversarial_demo.py

# Exportación y figuras (piloto v1)
uv run python scripts/export_results.py
make figures

# Campaña confirmatoria v2.1 (sustituye a lo anterior como resultado
# vigente -- congelada, RUN_COMPLETED/CLOSURE_VALID, no reproducible sin
# gastar API real: el archivo crudo ya está commiteado)
uv run python scripts/freeze_protocol_v2_1.py --verify
uv run python scripts/verify_tfm_closure_v2_1.py --final \
    --receipt-log data/protocol_v2_1/runs_v2/receipts_2.jsonl \
    --code-manifest-path data/protocol_v2_1/code_freeze_manifest.json \
    --report-path data/protocol_v2_1/confirmatory_report_v2_1_2.json

# Figuras del capítulo confirmatorio (§8.0)
uv sync --group figures
uv run python scripts/make_figures_v2_1.py
```

Anexo B. Artefactos de datos

Campaña confirmatoria v2.1 (vigente):

Fichero	Contenido
data/protocol_v2_1/runs_v2/confirmatory_observations_v21_2d36433e...jsonl	21.478 observaciones crudas, fila a fila
data/protocol_v2_1/code_freeze_manifest.json	Manifiesto congelado vigente (tfm-protocol-v2.1.2)
data/protocol_v2_1/code_freeze_manifest_v2_1_1.json	Manifiesto anterior, archivado por procedencia
data/protocol_v2_1/confirmatory_report_v2_1_2.json	Informe vigente — H1a-H8, 13 entradas, sin protocol_violation
data/protocol_v2_1/confirmatory_report_v2_1_1.json / confirmatory_report_v2_1_1_PRE_H2_FIX.json	Informes anteriores al arreglo de H2, conservados
reports/figures/v21_hypotheses_forest.{png,svg}	Las 9 pruebas, estimación e IC95, confirmada/no confirmada
reports/figures/v21_h4_categories.{png,svg}	Mutación no autorizada de C por las 7 categorías de H4
reports/figures/v21_h2_tokens.{png,svg}	Ahorro de tokens de C contra A y contra B

Piloto v1 (exploratorio, contexto):

Fichero	Contenido
<code>data/bench_v1.jsonl</code>	480 casos del benchmark
<code>data/freeze_manifest.json</code>	Hashes del protocolo congelado (schema 1.1)
<code>data/experiment_results.json</code>	Resumen histórico agregado (OpenRouter, parseo regalado)
<code>data/experiment_results_real_parser.json</code>	Referencia exploratoria agregada más reciente (Groq, parseo real + normalización)
<code>data/experiment_results_groq_given_args.json</code>	Réplica que separa proveedor de régimen (Groq, argumentos dados)
<code>data/retriever_comparison.json</code>	TF-IDF vs embeddings vs híbrido
<code>data/injecagent_stress_test_results.json</code>	Detección léxica, 510 payloads
<code>data/injection_resistance_results.json</code>	Resistencia por canal, 1.530 casos
<code>data/odoo_governed_demo_results.json</code>	Pipeline gobernado contra Odoo real
<code>data/odoo_adversarial_results.json</code>	Casos adversariales contra Odoo real
<code>data/real_requests_eval.json</code>	Recuperadores sobre 120 peticiones reales (validación de producto)
<code>data/real_requests_llm_eval.json</code>	Router LLM sobre las mismas peticiones, 120 llamadas reales
<code>data/router_designs_eval.json</code>	Cinco diseños de enrutado, calibrados en dev y juzgados held-out
<code>data/skill_profiles.json</code>	Descripciones enriquecidas, fuera del catálogo congelado
<code>data/annotation_review_sheet.csv</code>	Muestra estratificada para el segundo anotador — instrumento construido, paso retirado formalmente por v2.1, no completado
<code>data/real_requests.csv</code>	Peticiones reales — gitignorado , puede contener datos de clientes

Anexo C. Documentación técnica complementaria

`docs/results.md` (resultados completos con las cinco corridas), `docs/dataset-card.md`, `docs/experiment-protocol.md`, `docs/threat-model.md`, `docs/traceability-rubric.md`, `docs/retriever-comparison.md`, `docs/injecagent-stress-test.md`, `docs/odoo-demo.md`, `docs/audit.md` (registro completo de los quince defectos), `docs/spec-coverage.md` (cobertura §-por-§ de la especificación normativa), `docs/product-viability.md` (transferencia a producto: qué afirmación comercial sostiene cada número y cuál no), `docs/defensa.md` (estrategia de defensa: en qué orden contar los resultados y cómo responder las siete preguntas difíciles), `docs/presentacion.md` (15 diapositivas, contenido y locución) y `docs/video-guion.md` (guion literal del vídeo de 4 minutos).

Anexo D. Trabajo pendiente declarado

1. **Kappa de anotación** — retirado formalmente, no completado. El instrumento se construyó (`data/annotation_review_sheet.csv`) pero el paso humano no se va a ejecutar; el protocolo v2.1 sin anotación humana lo sustituye y ya está implementado, congelado y ejecutado (§6.3, §8.0).
2. **Workbook de Tableau** — insumos generados, montaje manual.
3. **Actualización de** `data/evidence_registry.json` **y** `src/erp_agent_os/claims.py` para que el contrato automático de afirmaciones distinga hipótesis confirmadas de no confirmadas de v2.1, en vez de su binario heredado de la era v1 (decisión de política pendiente, no una tarea de escritura).
4. **Vídeo de competición y presentación de defensa** — guion y contenido escritos (`docs/video-guion.md` , `docs/presentacion.md`); falta grabar y maquetar.